Causal Effect Engine — Project Brief

Thesis: Build a heterogeneous treatment-effect engine — a system that estimates not just *whether* an intervention works but *for whom*, validates those estimates honestly, and serves "who to treat" decisions through an API. The causal core is econometrics and measurement; the heterogeneous-effect layer is genuine ML (estimating a quantity you can never directly observe); the serving layer fills the ingestion/deployment/cloud gap. Few people do all three well.

This document is self-contained. You should be able to execute the whole project from this file alone.

---

1. What you're building

A research-grade causal estimation core, with a decision-product layer on top:

- **Research core:** ingest data, encode a causal DAG and its assumptions, estimate average and heterogeneous treatment effects with modern estimators, and stress-test the result with refutation and sensitivity analysis.
- **Decision layer:** rank units by their estimated treatment effect ("uplift") so you can target an intervention at the people it actually helps.
- **Serving:** expose it through a `/effect` endpoint (estimate an effect for a given configuration) and a `/recommend` endpoint (rank who to treat), containerized and deployed.

Two ways to lean, pick by taste — but the plan below builds the product flavor on top of a research-grade core so you get both stories:

- **Decision-product flavor:** emphasize uplift/targeting. More commercial, more obviously ML.
- **Research-tool flavor:** emphasize rigorous effect estimation from messy observational data. Closer to quant research and policy work.

---

2. Datasets

Dataset	Role in the project
LaLonde / NSW (job-training program)	Exists in both randomized and observational versions. Run your *observational* pipeline and show it recovers the *experimental* answer. This is your headline validation result.

Dataset	Role in the project
IHDP (Infant Health and Development Program, semi-synthetic)	True CATEs are known, so you can compute PEHE (precision in estimation of heterogeneous effects) and rank your CATE estimators properly.
Criteo Uplift (~25M rows, real ad uplift)	Demonstrate scale and the targeting product; evaluate with Qini/AUUC.
Hillstrom MineThatData email (optional, smaller)	A gentler uplift dataset to prototype on before Criteo.

Recommended path: validate on LaLonde + IHDP (ground truth available), then demonstrate scale/realism on Criteo.

---

3. The methods ladder

Climb these in order. The point is to show you understand *why* each rung exists, not to use the fanciest one.

0. Naive baseline. Compare treated vs. untreated outcome means. Build it first so you can later show how badly confounding corrupts it. It's your strawman.

1. Identification before estimation. Draw the causal DAG. State the assumptions explicitly: unconfoundedness/ignorability, positivity (overlap), SUTVA. This step *is* the econometric mindset and is what signals you're not just throwing models at data.

2. Average treatment effect (ATE). Climb through the estimators to feel their tradeoffs:

- Regression adjustment
- Propensity-score methods: IPW, matching, stratification
- Doubly-robust (AIPW): consistent if *either* the outcome model *or* the propensity model is right
- Double/Debiased ML (DML): ML nuisance models with orthogonalization and cross-fitting for valid inference. The modern workhorse and the clean bridge between econometrics and ML.

3. Heterogeneous effects (CATE). The genuinely ML layer:

- Meta-learners: S / T / X / R / DR learners
- Causal forests
- DML with treatment–feature interactions This layer powers the targeting product.

4. When unconfoundedness fails (optional bonus modules). If the data has the structure for it:

- Difference-in-differences (panel data)
- Instrumental variables
- Regression discontinuity Squarely in the econometrics wheelhouse; makes the project feel complete.

---

4. Evaluation — the heart of the project

The fundamental problem of causal inference: you never observe both potential outcomes for the same unit, so you **cannot** just hold out a test set and compute error on real data. A model can predict outcomes well and estimate effects terribly. Spend disproportionate effort here — it is what separates a serious project from a toy.

Four honest strategies; use several together:

1. **Benchmark against experimental ground truth.** On LaLonde, show your observational pipeline recovers the RCT answer. Landing near the experimental benchmark is a genuine mic-drop portfolio result.
2. **Semi-synthetic data with known effects.** On IHDP/ACIC, compute PEHE to rank CATE estimators.
3. **Refutation and sensitivity on real data.** You can't prove correctness, but you can stress-test: placebo-treatment tests, adding a random common cause, subset-stability, and sensitivity analysis quantifying how strong an unobserved confounder would need to be to overturn the result (E-values; the sensemakr / Cinelli–Hazlett approach). "Our effect survives unless a confounder is stronger than X" is how credible causal work is communicated.
4. **Uplift-specific metrics.** For the targeting layer: Qini curves, AUUC, uplift-by-decile.

---

5. Tech stack

Layer	Tool	Why
Identification + refutation	DoWhy	Enforces model → identify → estimate → refute; built-in refutation tests
Estimators (DML, forests, meta-learners)	EconML	Industrial-grade CausalForestDML, DRLearner, LinearDML, etc.
Uplift + Qini/AUUC	CausalML (Uber)	Meta-learners plus uplift evaluation metrics
IV / panel / DiD	linearmodels, statsmodels	Econometrics comfort zone for the bonus modules

Layer	Tool	Why
Base ML learners	scikit-learn, LightGBM	Nuisance models inside DML/meta-learners
DAG encoding	NetworkX + graphviz	Express and visualize assumptions
Storage / ingestion	DuckDB + pandas	SQL on local files; fills the ingestion gap
Serving	FastAPI + Docker	/effect and /recommend endpoints, containerized
Demo UI (optional)	Streamlit	Upload data, specify DAG, see effects — good for a portfolio link
Deployment	Railway / Render free tier	Cloud exposure without AWS overhead yet
Testing	pytest	Unit tests on estimators + refutation logic
Config	pydantic-settings + YAML	Type-safe config; no magic numbers in code

6. Repo structure

```
causal-effect-engine/
│
├── data/
│   ├── raw/                    # LaLonde, IHDP, Criteo, etc.
│   ├── processed/
│   └── db/                     # DuckDB file
│
├── notebooks/
│   ├── 01_eda_overlap.ipynb        # Outcome EDA + propensity overlap checks
│   ├── 02_ate_ladder.ipynb         # Naive → IPW → AIPW → DML comparison
│   ├── 03_cate_models.ipynb        # Meta-learners vs. causal forest
│   └── 04_validation.ipynb         # LaLonde benchmark + PEHE + refutation
│
├── src/
│   ├── ingestion/
│   │   └── load_datasets.py        # Pull/cache datasets → DuckDB
│   ├── identification/
│   │   ├── dag.py                  # Define DAG, encode assumptions
│   │   └── checks.py               # Positivity/overlap, balance diagnostics
```

```
|   ├── estimators/
|   |   ├── ate.py                  # IPW, matching, AIPW, DML wrappers
|   |   ├── cate.py                 # Meta-learners, causal forest
|   |   └── econometric.py          # IV, DiD, RDD (bonus)
|   ├── evaluation/
|   |   ├── benchmark.py            # Observational-vs-experimental (LaLonde)
|   |   ├── pehe.py                 # Semi-synthetic ground-truth error
|   |   ├── refutation.py           # Placebo, random cause, subset tests
|   |   ├── sensitivity.py          # E-values / unobserved-confounder bounds
|   |   └── uplift.py               # Qini, AUUC, uplift-by-decile
|   └── serve/
|       └── api.py                  # FastAPI: /effect, /recommend
|
├── tests/                          # Unit tests on estimators + refutation
logic
├── configs/
|   └── config.yaml                 # Dataset, treatment, outcome,
confounders, DAG
├── Dockerfile
├── requirements.txt
└── research_memo.md                # Question → identification → results →
sensitivity
```

---

7. Build order / milestones

1. **Ingestion + EDA with an overlap lens.** Load LaLonde into DuckDB; look at propensity-score overlap between treated/untreated. Poor overlap is the silent killer — confront it on day one.
2. **DAG + identification module.** Encode assumptions explicitly before any estimation.
3. **The ATE ladder.** Naive → IPW → AIPW → DML, side by side, on the same data. The comparison is the point.
4. **The LaLonde benchmark.** Show the observational DML pipeline recovers the experimental effect. This is your headline — get it working early.
5. **CATE layer.** Meta-learners and a causal forest; validate with PEHE on IHDP.
6. **Refutation + sensitivity suite.** Placebo tests, random common cause, E-values.
7. **Uplift + targeting on Criteo.** Demonstrate scale and the decision product; evaluate with Qini/AUUC.
8. **Serve it.** FastAPI `/effect` and `/recommend`, containerized and deployed.
9. **Research memo.** Written like a credible empirical paper: question, identification strategy, estimates, robustness, limitations.

Suggested first week (a concrete, motivating start)

- **Day 1:** Create the repo with the structure above; set up the environment; pull LaLonde into DuckDB; plot the propensity-score overlap.
- **Days 2–3:** Build the DAG/identification module and the naive baseline.
- **Days 4–5:** Implement the ATE ladder (IPW, AIPW, DML).
- **End-of-week milestone:** Your observational DML estimate vs. the experimental benchmark, written up in one page. A real, validating result in seven days.

---

8. Pitfalls checklist

- **Bad controls / colliders.** Don't "control for everything." Conditioning on a post-treatment variable, mediator, or collider *induces* bias. A worked example of a control that *hurts* the estimate is a sophisticated touch.
- **Positivity violations.** Near-zero or near-one propensities make IPW weights explode. Diagnose and trim; don't ignore.
- **Prediction ≠ identification.** A high-R^2 outcome model says nothing about whether your effect is unbiased. Keep the two scorecards separate.
- **Over-trusting unconfoundedness.** The honest stance: "here's how strong a hidden confounder would need to be to change the conclusion." That's what the sensitivity module delivers.

---

9. Why this is also a quant-research muscle

Treatment-effect estimation is structurally the same problem as event studies (did this event *cause* the return move?), alpha/attribution analysis (is this signal causal or just correlated with a known factor?), and policy/regime analysis. Double ML shows up in modern empirical finance precisely because it gives valid inference with high-dimensional, ML-modeled nuisance. The project builds a quant-relevant skill while being a complete, deployable data-and-AI artifact.

---

10. Definition of "showcase-ready"

- Observational pipeline recovers the experimental benchmark within a reasonable confidence interval (LaLonde).
- CATE models validated on IHDP via PEHE.
- Refutation + sensitivity suite implemented and reported.
- Uplift demo on Criteo with a Qini curve.
- Deployed API with working `/effect` and `/recommend` endpoints.

- A 2–4 page research memo.
- Clean repo: tests, a README, reproducible config, no magic numbers.

---

11. Reading list (canonical references)

Books:

- Cunningham, *Causal Inference: The Mixtape*
- Huntington-Klein, *The Effect*
- Hernán & Robins, *Causal Inference: What If*
- Pearl, *The Book of Why* (intuition) / *Causality* (formal)

Papers / methods:

- Rosenbaum & Rubin (1983) — propensity scores
- LaLonde (1986); Dehejia & Wahba (1999) — the NSW benchmark and its propensity-score reanalysis
- Hill (2011) — BART for causal inference / IHDP
- Chernozhukov et al. (2018) — Double/Debiased Machine Learning
- Wager & Athey (2018) — causal forests
- Künzel et al. (2019) — meta-learners (X-learner)
- VanderWeele & Ding — the E-value for sensitivity analysis
- Cinelli & Hazlett (2020) — sensitivity analysis (sensemakr)

(Confirm exact titles/years as you go; these are well-known works but worth checking against the source.)